

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Компьютерные науки и анализ данных»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Программный проект на тему:
Веб-приложение для группового общения

Выполнил студент:

группы #БКНАД221, 4 курса

Деев Семен Алексеевич

Принял руководитель ВКР:

Промыслов Валентин Валерьевич

Доцент

Факультет компьютерных наук НИУ ВШЭ

Соруководитель:

Овчинников Сергей Андреевич

Ведущий инженер

ООО "ВымпелКом-Информационные технологии"

Содержание

Аннотация	4
1 Введение	5
2 Анализ существующих приложений	7
2.1 Discord	8
2.2 Zoom	9
2.3 Jitsi Meet	10
2.4 Matrix / Element	10
2.5 TeamSpeak	11
Сводная таблица	12
3 Формирование требований к приложению	14
3.1 Функциональные требования	14
3.2 Нефункциональные требования	17
4 Архитектура и проектирование	19
4.1 Общая архитектура	19
4.2 Модель данных	19
4.3 Backend-архитектура	20
4.4 WebSocket и доставка сообщений	20
4.5 WebRTC и голосовые топики	21
4.6 Frontend-архитектура	22
5 Реализация	22
5.1 Реализация backend	22
5.2 Реализация frontend	24
5.3 Локальная инфраструктура	25
6 Тестирование и оценка результата	26
6.1 Подход к тестированию	26
6.2 Сценарии функционального тестирования	26
6.3 Оценка производительности	27
6.4 Оценка соответствия требованиям	27

7 Заключение	28
Список литературы	30

Аннотация

В выпускной квалификационной работе рассматривается проектирование и реализация веб-приложения «Chatterly», предназначенного для группового общения пользователей в рамках онлайн-сообществ. Актуальность работы обусловлена потребностью в доступных браузерных коммуникационных сервисах, которые объединяют постоянные сообщества, тематические каналы, историю переписки и групповые звонки, но не требуют установки отдельного клиента и не обладают избыточной сложностью интерфейса.

Целью работы является разработка прототипа веб-приложения для организации пользовательских сообществ, личной переписки, обмена текстовыми сообщениями в реальном времени и проведения аудио- и видеозвонков. В ходе работы проведён анализ существующих решений, сформированы функциональные и нефункциональные требования, спроектирована архитектура системы и реализованы основные компоненты приложения.

Серверная часть приложения реализована на языке Go, клиентская часть – с использованием JavaScript и SolidJS. Для хранения данных применяется PostgreSQL, для сессий, кэширования и публикации событий – Redis, для работы с пользовательскими изображениями – S3-совместимое хранилище. Обмен событиями в реальном времени реализован на основе WebSocket, а аудио- и видеосвязь – на основе WebRTC.

Результатом работы является прототип коммуникационного веб-приложения, включающий управление пользователями, серверы и топики, личные сообщения, историю переписки, доставку сообщений в реальном времени и базовую поддержку групповых звонков. В работе также рассмотрены ограничения текущей реализации и направления дальнейшего развития системы.

Ключевые слова

Веб-приложение, групповая коммуникация, обмен сообщениями в реальном времени, видеосвязь, Go, JavaScript, PostgreSQL, Redis, WebSocket, WebRTC.

1 Введение

Современные цифровые сообщества в значительной степени опираются на программные средства сетевой коммуникации. Такие средства используются в образовательных группах, профессиональных командах, игровых сообществах, клубах по интересам и иных объединениях, где пользователям необходимо не только обмениваться отдельными сообщениями, но и поддерживать устойчивую структуру общения. Для подобных сценариев важны постоянные пространства взаимодействия, тематическое разделение обсуждений, история переписки, оперативная доставка новых сообщений, а также возможность голосовой и видеосвязи.

Развитие веб-технологий привело к тому, что значительная часть коммуникационных функций может быть реализована непосредственно в браузере. Пользовательский сценарий в этом случае становится проще: для доступа к системе достаточно открыть веб-страницу, пройти регистрацию или авторизацию и перейти к нужному сообществу или диалогу. Такой подход особенно важен для пользователей, которым требуется быстрое подключение к общению без установки отдельного клиента и сложной настройки окружения.

Одним из наиболее близких по назначению решений является Discord, предоставляющий модель серверов, каналов, личных сообщений и голосовых комнат [4]. Однако использование Discord как универсальной основы для целевой аудитории ограничено двумя обстоятельствами. Во-первых, интерфейс и система настроек платформы могут быть избыточными для пользователей, которым необходим базовый набор функций группового общения. Во-вторых, доступ к Discord на территории Российской Федерации ограничен [10]. Следовательно, несмотря на функциональную близость к целевой модели, Discord не устраняет полностью рассматриваемую практическую проблему.

Другие распространённые решения также закрывают только отдельные части задачи. Zoom и Jitsi Meet удобны для видеоконференций и разовых встреч, но не ориентированы на модель постоянных сообществ с тематическими каналами и долговременной историей обсуждений [7, 2]. TeamSpeak хорошо решает задачу голосового общения, однако в меньшей степени соответствует современным требованиям к браузерному интерфейсу, видеосвязи и текстовому взаимодействию [5]. Matrix и Element предоставляют развитую модель независимой коммуникационной инфраструктуры и контроля данных, но могут быть сложнее для массового пользователя с точки зрения настройки и освоения [6, 1]. Таким образом, существующие аналоги либо недоступны, либо сложны, либо ориентированы на более узкий коммуникационный сценарий.

Актуальность настоящей работы определяется необходимостью разработки доступного веб-приложения, которое объединяет базовые возможности современных коммуникационных платформ: создание сообществ, разделение общения по тематическим каналам, личные сообщения, хранение истории, обмен событиями в реальном времени и групповые аудио-/видеозвонки. При этом особое значение имеет простота пользовательского интерфейса, поскольку целевое приложение должно быть пригодно не только для технически подготовленных пользователей, но и для обычных участников онлайн-сообществ.

Объектом работы является веб-приложение для группового общения пользователей. Предметом работы являются архитектурные и программные решения, необходимые для реализации такого приложения: модель данных, серверная бизнес-логика, клиентский интерфейс, механизм доставки событий в реальном времени и механизм установления мультимедийных соединений между участниками звонка.

Целью выпускной квалификационной работы является проектирование и реализация прототипа веб-приложения «Chatterry», предназначенного для организации группового общения пользователей в рамках сообществ, структурированных по тематическим топикам.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 провести анализ существующих коммуникационных приложений и определить их ограничения относительно целевого сценария;
- 2 сформировать функциональные и нефункциональные требования к разрабатываемой системе;
- 3 спроектировать архитектуру серверной части, клиентской части, базы данных, WebSocket-контура и подсистемы аудио-/видеозвонков;
- 4 реализовать серверную часть приложения на языке Go с REST API, WebSocket-интерфейсом, доступом к PostgreSQL, Redis и S3-совместимому хранилищу;
- 5 реализовать клиентскую часть приложения на JavaScript и SolidJS, включая страницы авторизации, личных сообщений, серверов, текстовых и голосовых топиков;
- 6 реализовать обмен текстовыми сообщениями в реальном времени и сохранение истории сообщений;
- 7 реализовать базовую поддержку групповых аудио- и видеозвонков с использованием WebRTC;

8 описать подход к тестированию и оценить соответствие полученного прототипа сформированным требованиям.

Практическая значимость работы состоит в создании прототипа независимого веб-приложения, которое может использоваться как основа для дальнейшего развития коммуникационной платформы. В отличие от решений, ориентированных только на видеовстречи или только на голосовое общение, разрабатываемая система объединяет несколько связанных пользовательских сценариев: постоянные сообщества, тематические топики, личную переписку, историю сообщений и звонки. В отличие от функционально насыщенных коммерческих платформ, Chatterry рассматривается как более простой по пользовательской модели прототип, в котором основное внимание уделяется базовым сценариям группового взаимодействия.

Техническая значимость работы связана с интеграцией нескольких механизмов, характерных для современных приложений реального времени. Серверная часть реализована на языке Go и построена по слоистой архитектуре: выделены доменные модели, сервисы бизнес-логики, адаптеры к внешним хранилищам и HTTP/WebSocket API. Для хранения постоянных данных используется PostgreSQL, для кэширования сессий и публикации событий – Redis, для пользовательских изображений – S3-совместимое объектное хранилище. Доставка событий между пользователями реализована на основе WebSocket, а передача аудио- и видеопотоков – на основе WebRTC.

В результате работы разработан прототип приложения, включающий регистрацию и аутентификацию пользователей, управление профилем, создание серверов и топиков, личные сообщения, текстовые сообщения в топиках, постраничную загрузку истории, уведомления о новых сообщениях и пользовательский интерфейс голосовых топиков. Также реализована серверная обработка событий звонков, включая подключение участников, передачу сигнальных сообщений и обработку медиапотоков.

2 Анализ существующих приложений

Для обоснования разработки Chatterry недостаточно сравнить аналоги только по принципу наличия или отсутствия отдельных функций. Коммуникационные приложения отличаются не только набором возможностей, но и основной моделью взаимодействия: одни строятся вокруг постоянных сообществ, другие – вокруг разовой встречи, третьи – вокруг федеративной инфраструктуры или голосового канала. Поэтому в данном разделе рассматриваются как пользовательские функции, так и архитектурные следствия выбранной модели.

Сравнение проводится по следующим критериям:

- 1 основная пользовательская модель: постоянное сообщество, личная переписка, встреча, комната или голосовой канал;
- 2 структура коммуникации: наличие серверов, тематических текстовых и голосовых пространств, ролей и участников;
- 3 поддержка сообщений в реальном времени, истории переписки и уведомлений;
- 4 поддержка групповой аудио- и видеосвязи, демонстрации экрана и браузерного подключения;
- 5 модель развёртывания и контроля данных: централизованный сервис, самостоятельное развёртывание или федерация;
- 6 сложность освоения и доступность для целевой аудитории.

Такой набор критериев позволяет связать анализ аналогов с последующими требованиями к Chatterry: если аналог закрывает отдельный сценарий, но не подходит как целостная основа, из этого формулируется конкретное проектное решение для разрабатываемой системы.

2.1 Discord

Discord является наиболее близким функциональным аналогом Chatterry. Его основная пользовательская модель строится вокруг сервера как постоянного цифрового пространства, в котором участники объединяются по интересам, учёбе, работе или другой общей цели. Внутри сервера создаются каналы разных типов: текстовые каналы для обсуждений, голосовые каналы для общения и видеосвязи, а также дополнительные формы организации обсуждений, например категории и треды. Для крупных сообществ Discord также предоставляет роли, права доступа, модерацию, приглашения, onboarding и другие средства управления участниками [4].

Сильная сторона Discord состоит в том, что структура общения совпадает с естественной структурой многих онлайн-сообществ: есть общее пространство, внутри него – тематические каналы, а пользователи могут переходить между текстовым и голосовым взаимодействием. Для проектируемого приложения эта модель важна как предметный ориентир: Chatterry также должен иметь сущности пользователя, сервера, участника, текстового топика, голосового топика и сообщений.

Ограничения Discord связаны не с отсутствием функций, а с применимостью продукта к целевому сценарию. Во-первых, развитая система ролей, настроек, интеграций и дополнительных инструментов повышает сложность интерфейса для пользователя, которому требуется только базовое групповое общение. Во-вторых, сервис является централизованной внешней платформой, поэтому пользователь и организация не контролируют развёртывание и хранение данных. В-третьих, доступ к Discord в Российской Федерации был ограничен Роскомнадзором [10]. Следовательно, для Chatterry целесообразно использовать продуктивную идею серверов и каналов, но реализовать её как более простой независимый MVP с минимальным набором ролей и сценариев.

2.2 Zoom

Zoom ориентирован прежде всего на видеоконференции, онлайн-встречи и вебинары. Важными возможностями платформы являются подключение участников к встрече, аудио- и видеосвязь, демонстрация экрана, управление участниками и инструменты для корпоративной коммуникации [7]. В составе Zoom Workplace также существует Team Chat: он поддерживает личные сообщения, групповые чаты и каналы, которые могут использоваться как долгосрочные пространства для проектов или команд [3].

Таким образом, Zoom нельзя корректно описывать как систему, в которой полностью отсутствует текстовое взаимодействие. Однако его ключевая модель всё равно отличается от целевого сценария Chatterry. Zoom исторически и продуктово строится вокруг встречи или корпоративной рабочей среды, а не вокруг пользовательского сообщества, где текстовые топики, голосовые комнаты, история сообщений и членство в сервере являются равноправными частями одной предметной модели. Каналы Zoom Team Chat полезны для рабочих групп внутри аккаунта организации, но они не образуют модель пользовательских серверов, аналогичную Discord.

Для Chatterry из анализа Zoom следует два вывода. Первый вывод состоит в том, что видеосвязь должна иметь низкий барьер входа и поддерживать привычные действия пользователя: включение микрофона, камеры и демонстрации экрана. Второй вывод состоит в том, что звонок не должен становиться единственной центральной сущностью приложения. В Chatterry аудио- и видеосвязь должны быть встроены в постоянное сообщество и запускаться в контексте голосового топики, рядом с текстовыми топиками и историей сообщений. Для российской аудитории также сохраняется общий риск зависимости от внешнего коммерческого сервиса [9].

2.3 Jitsi Meet

Jitsi Meet представляет собой открытое решение для аудио- и видеоконференций. Сервис позволяет создавать встречу по ссылке, подключать участников без обязательной регистрации, демонстрировать рабочий стол, использовать встроенный чат и при необходимости разворачивать собственный экземпляр системы [2]. По сравнению с коммерческими видеосервисами Jitsi особенно важен как пример браузерной доступности и самостоятельного развёртывания.

Архитектурно Jitsi Meet решает задачу медиа-сеанса, а не задачу длительного сообщества. Центральной сущностью является встреча с URL, к которой подключаются участники. Такая модель хорошо подходит для быстрого звонка, но в ней нет полноценной предметной области, необходимой Chatterry: серверов пользователя, набора постоянных текстовых и голосовых топиков, истории сообщений по каждому топик, профилей участников и личных диалогов. Встроенный чат Jitsi полезен внутри встречи, но не заменяет долгосрочную переписку сообщества.

Из Jitsi Meet для Chatterry следует сохранить идею работы через браузер и минимального барьера подключения к звонку. При этом звонок должен быть не самостоятельной одноразовой комнатой, а частью постоянного пространства общения. Поэтому в Chatterry медиасоединение реализуется через WebRTC, а сигнальные события и состояние участников привязываются к голосовому топик и доставляются через общий real-time контур приложения.

2.4 Matrix / Element

Matrix отличается от остальных рассмотренных решений тем, что является не отдельным приложением, а открытым протоколом для децентрализованной коммуникации. Спецификация Matrix описывает открытые API для обмена сообщениями, VoIP-сигналикации и синхронизации данных в федерации серверов. В этой модели пользователь работает через клиент, клиент обращается к домашнему серверу, а история и состояние комнат синхронизируются между серверами-участниками [6]. Element является одним из наиболее известных клиентов и платформенных решений на основе Matrix; он ориентирован на защищённую коммуникацию, контроль данных, сквозное шифрование, развёртывание в облаке или on-premise [1, 8].

Сильная сторона Matrix / Element состоит в независимости от единого поставщика и в формализованной архитектуре обмена событиями. Для Chatterry особенно важны идеи ком-

нат, событий, профилей, истории сообщений и разделения клиент-серверного API от межсерверного взаимодействия. В то же время федеративная архитектура существенно повышает сложность реализации: необходимо учитывать синхронизацию состояния между серверами, идентичность пользователей в разных доменах, устройства, ключи шифрования, согласование истории и обработку конфликтов.

Для прототипа Chatterly такая сложность избыточна. Цель работы состоит не в реализации открытого федеративного протокола, а в создании независимого веб-приложения для базовых сценариев группового общения. Поэтому из Matrix / Element следует заимствовать не федерацию как обязательное требование, а архитектурный принцип событийной коммуникации и контроля данных. В MVP Chatterly это выражается в централизованной серверной части, собственной базе данных, WebSocket-событиях для доставки изменений и возможности дальнейшего развития системы без зависимости от конкретной зарубежной платформы.

2.5 TeamSpeak

TeamSpeak ориентирован прежде всего на голосовое общение. Базовый пользовательский сценарий состоит в подключении к серверу и переходе в канал, где участники могут разговаривать друг с другом; голосовые данные передаются через сервер [5]. Такая модель исторически хорошо подходит для игровых и технических сообществ, которым важны постоянные голосовые комнаты, качество аудио и управление доступом.

Основное ограничение TeamSpeak – узкая направленность. В Chatterly текстовые сообщения, личные диалоги, история и звонки должны быть частью единого браузерного приложения. TeamSpeak в первую очередь решает задачу голосовых каналов. Даже при наличии дополнительных средств общения его основная предметная модель не делает текстовые топики, историю сообщений и видеосвязь равноправными объектами системы.

Для Chatterly TeamSpeak подтверждает важность постоянных голосовых комнат: пользователю удобно не создавать новую встречу каждый раз, а заходить в уже существующий голосовой топик внутри сообщества. Однако эта идея должна быть реализована в современной веб-модели: пользователь открывает приложение в браузере, выбирает сервер и голосовой топик, а передача медиа выполняется через WebRTC при сохранении связи с остальными компонентами системы.

Сводная таблица

Решение	Основная модель	Полезные свойства	Ограничения для целевого сценария	Вывод для Chatterly
Discord	постоянные серверы с каналами, ролями и участниками	серверы, текстовые и голосовые каналы, видеосвязь, права доступа, модерация	зависимость от внешней централизованной платформы, высокая насыщенность интерфейса, ограниченная доступность в РФ	использовать модель серверов и топиков, но упростить MVP и сохранить независимость реализации
Zoom	встреча и корпоративная рабочая среда; Team Chat поддерживает каналы	устойчивый сценарий видеовстреч, демонстрация экрана, быстрый вход в звонок, рабочие чат-каналы	каналы не образуют модель пользовательских серверов с текстовыми и голосовыми топиками; звонок остаётся центральным сценарием	встроить аудио- и видеосвязь в постоянное сообщество, а не строить приложение только вокруг встречи
Jitsi Meet	браузерная встреча по ссылке	открытый код, простое подключение, самостоятельное развёртывание, встроенный чат во время звонка	нет полноценной модели серверов, профилей, топиков и долговременной истории сообщений	использовать браузерную доступность звонков, но связать WebRTC с постоянными топиками и историей
Matrix / Element	федерация домашних серверов, комнаты и события	открытый протокол, контроль данных, сквозное шифрование, on-premise и cloud-развёртывание	высокая архитектурная и пользовательская сложность для MVP: федерация, синхронизация, устройства и ключи	сохранить событийную модель и контроль данных, но реализовать централизованный прототип
TeamSpeak	серверы и голосовые каналы	постоянные голосовые комнаты, управление доступом, ориентация на низкую задержку аудио	голосовое общение является главным сценарием; текстовая история и видеосвязь не являются равноправной основой продукта	реализовать голосовые топиксы внутри веб-приложения вместе с текстовыми сообщениями и видеосвязью

Таблица 2.1: Сравнение существующих решений по модели взаимодействия и выводам для проекта

Сравнение в таблице 2.1 показывает, что ни одно из рассмотренных решений не закрывает целевой сценарий полностью. Из проведённого анализа следует, что Chatterly должен объединить несколько свойств в одной системе: постоянные пользовательские серверы, текстовые и голосовые топиксы, личные сообщения, сохранение истории, доставку событий в реальном времени и браузерные аудио- и видеозвонки. На уровне архитектуры это приводит к выбору централизованного клиент-серверного MVP: PostgreSQL используется для постоянного состояния и истории, WebSocket – для доставки сообщений и событий звонков, WebRTC – для передачи аудио- и видеопотоков, а пользовательский интерфейс ограничивается базо-

выми сценариями без сложной системы ролей и федерации. Эти выводы используются далее при формировании требований к приложению.

3 Формирование требований к приложению

Требования к Chatterry формируются на основе анализа аналогов. Функциональные требования описывают поведение системы и пользовательские возможности. Нефункциональные требования фиксируют ограничения и качественные характеристики. Для удобства дальнейших ссылок требования имеют идентификаторы **F** для функциональных требований и **NF** для нефункциональных требований.

Вывод из анализа	Где проявляется	Требования	Архитектурное следствие
Необходима модель постоянных сообществ, а не только разовых встреч	Discord, Zoom, Jitsi Meet	F-3, F-4, NF-2	выделяются сущности сервера, участника и топика; структура сервера хранится в PostgreSQL
Текстовое общение, звонки и история должны быть частью одной системы	Discord, Zoom, Jitsi Meet, TeamSpeak	F-5, F-6, F-7, F-9	сообщения сохраняются в БД, а звонки привязываются к голосовым топикам
Для пользовательского опыта важна доставка событий в реальном времени	Discord, Matrix / Element, все чаты	F-8, F-10, NF-4	используется единый WebSocket endpoint и Redis pub/sub для доставки событий пользователям
Браузерная аудио- и видеосвязь должна быть встроена в приложение	Discord, Zoom, Jitsi Meet	F-9, F-10, NF-1	медиапоток передается через WebRTC, а сигнальные сообщения обрабатываются сервером
Сложные роли, федерация и избыточные настройки не требуются для MVP	Discord, Matrix / Element	F-3, F-11, NF-2, NF-5	выбирается централизованная архитектура с ролями owner/member и простым пользовательским сценарием
Система должна контролировать данные и доступ к действиям	Matrix / Element, Discord	F-1, F-2, NF-3, NF-6	данные хранятся в PostgreSQL и S3-совместимом хранилище; доступ проверяется на серверной стороне

Таблица 3.1: Прослеживаемость требований от анализа аналогов и выводов преддипломной практики

3.1 Функциональные требования

Функциональные требования ниже описывают не только пользовательскую возможность, но и ожидаемое поведение серверной и клиентской частей. Это необходимо для последующей проверки реализации и для устранения недостатка преддипломной практики, где требования были сформулированы без достаточной детализации сценариев и API.

F-1 Регистрация и аутентификация пользователей. Система должна позволять со-

здать учётную запись по имени пользователя, логину и паролю, выполнить вход, выход и получить сведения о текущем пользователе. Серверная часть должна проверять уникальность логина и имени пользователя, хранить пароль в виде хэша, создавать сессию после успешного входа и передавать клиенту session cookie. На уровне API этому соответствуют операции создания пользователя, входа, выхода и получения текущей сессии.

F-2 Управление пользовательским профилем. Пользователь должен иметь возможность просматривать текущий профиль, изменять имя, логин и пароль, удалять профиль, загружать и удалять аватар. При изменении логина или пароля сервер должен требовать текущий пароль. Пользовательское изображение должно храниться во внешнем S3-совместимом хранилище; при отсутствии аватара система должна генерировать identicon, чтобы пользователь оставался визуально различимым в списках и сообщениях.

F-3 Сообщества и участники. Приложение должно поддерживать создание сервера как постоянного пространства общения, получение списка серверов пользователя, поиск серверов, к которым пользователь ещё не присоединился, вступление в сервер и выход из него. Пользователь, создавший сервер, получает роль владельца; остальные участники получают роль member. Изменение и удаление сервера должны быть доступны только владельцу. В рамках MVP серверы рассматриваются как доступные через поиск пространства, без отдельной модели приватных приглашений.

F-4 Тематические топики. Внутри сервера должны поддерживаться топики двух типов: текстовый топик для сообщений и голосовой топик для звонков. Владелец сервера должен иметь возможность создавать, изменять и удалять топики. Система должна запрещать дублирование топиков с одинаковым именем и типом внутри одного сервера, а также должна проверять, что пользователь имеет доступ к серверу перед входом в топик.

F-5 Личные сообщения. Пользователь должен иметь возможность искать других пользователей, с которыми у него ещё нет личного диалога, создавать личный диалог, просматривать список диалогов и переходить к переписке. В списке диалогов должны отображаться собеседник, последнее сообщение и признак непрочитанности. Система должна запрещать создание диалога пользователя с самим собой и повторное создание уже существующего DM.

- F-6 Отправка и хранение текстовых сообщений.** Пользователь должен иметь возможность отправлять сообщения в личный диалог и текстовый топик. Сервер должен проверять доступ пользователя к соответствующему DM или топику, сохранять сообщение в PostgreSQL, обновлять состояние последнего сообщения и передавать событие участникам через real-time контур. Для отправки используются отдельные REST-операции для DM и сообщений текстового топика.
- F-7 История сообщений и пагинация.** Чат должен поддерживать загрузку первой страницы истории и последующих страниц при прокрутке вверх. Пагинация должна быть стабильной при наличии сообщений с одинаковым временем создания, поэтому cursor должен учитывать идентификатор чата или топика, время создания и идентификатор последнего загруженного сообщения. Это требование обеспечивает постоянный контекст общения, которого недостаточно в сервисах, ориентированных только на разовые встречи.
- F-8 WebSocket-подписки и уведомления.** Клиент должен подключаться к единому WebSocket endpoint `/ws/`. Соединение должно поддерживать события `join`, `leave`, `ping`, `pong`, `message` и `error`, а также каналы типов `dm`, `text_topic` и `voice_topic`. Перед подпиской сервер должен проверять доступ пользователя к каналу. При входящем личном сообщении интерфейс должен обновлять список диалогов, признак непрочитанности и показывать уведомление, если пользователь находится в другом чате.
- F-9 Голосовые и видеозвонки.** Пользователь должен иметь возможность подключаться к голосовому топику сервера и участвовать в групповом звонке. Клиентская часть должна поддерживать передачу микрофона, камеры и демонстрации экрана через WebRTC, отображение локального preview, плитки удалённых участников, выбор устройств и обработку ошибок доступа к media devices.
- F-10 WebRTC signaling и состояние голосового топика.** Для установления WebRTC-соединений сервер и клиент должны обмениваться сигнальными событиями семейства `voice_*`: `offer`, `answer`, `ICE candidates`, состояние участников, вход и выход пользователя из голосового топика. Серверная часть должна хранить состояние активных участников, обрабатывать отключение пользователя и рассылать изменения остальным участникам звонка.
- F-11 Пользовательский интерфейс основных сценариев.** Клиентская часть должна предоставлять страницы регистрации и входа, список личных диалогов, поиск пользо-

вателей, список серверов, поиск и создание серверов, управление сервером, переходы между текстовыми и голосовыми топиками. Раздел личных сообщений должен включать экран выбора диалога, поиск собеседника и страницу чата. Раздел серверов должен включать создание, управление, редактирование, текстовый топик и голосовой топик.

F-12 Обновление интерфейса без ручной перезагрузки. После создания, изменения или удаления серверов, топиков, диалогов и сообщений интерфейс должен обновлять соответствующие списки и текущий экран без ручной перезагрузки страницы. Это требование связано с пользовательской моделью приложений реального времени и отличается Chattery от статических интерфейсов, где пользователь должен вручную обновлять состояние.

3.2 Нефункциональные требования

Нефункциональные требования определяют условия, при которых функциональные возможности должны работать приемлемо для пользователя и быть пригодными для дальнейшего развития проекта.

NF-1 Доступность через браузер. Приложение должно работать как веб-система и не требовать установки отдельного desktop-клиента. Это требование следует из ограничений TeamSpeak и из сильных сторон Jitsi Meet: пользователь должен открыть приложение в браузере, пройти аутентификацию и получить доступ к чатам и звонкам.

NF-2 Простота пользовательского сценария. Основные действия должны выполняться с небольшим числом шагов: регистрация, вход, поиск пользователя, создание DM, создание сервера, вступление в сервер, переход в топик, отправка сообщения и подключение к звонку. В MVP не вводится сложная система ролей, федерации и расширенных настроек, поскольку анализ Discord и Matrix / Element показал, что такая функциональность повышает порог входа.

NF-3 Сохранность и разделение данных. Пользователи, серверы, участники, топик, личные сообщения и сообщения топиков должны храниться в PostgreSQL. Пользовательские изображения должны храниться в S3-совместимом объектном хранилище. Redis должен использоваться для сессий, доставки событий и временного состояния звонков, но не как единственное хранилище постоянной истории общения.

- NF-4 **Производительность real-time взаимодействия.** MVP должен поддерживать не менее десяти активных пользователей в одном чате или звонке без заметной пользовательской задержки. Для текстового real-time взаимодействия используются WebSocket и Redis pub/sub, для истории сообщений – индексы и cursor-based пагинация, для медиа – WebRTC. Проверка данного требования должна выполняться в разделе тестирования через сценарии одновременной отправки сообщений и подключения нескольких участников к голосовому топику.
- NF-5 **Масштабируемость архитектуры.** Архитектура должна допускать дальнейшее увеличение числа пользователей, сообщений и WebSocket-соединений. Для этого REST API, WebSocket manager, сервисы бизнес-логики, PostgreSQL, Redis и S3-совместимое хранилище должны быть разделены как самостоятельные компоненты. Real-time события должны доставляться через Redis-каналы пользователя, а состояние голосового топики должно учитывать ownership backend-узла, что оставляет возможность запуска нескольких экземпляров серверной части.
- NF-6 **Безопасность и контроль доступа.** Пароли должны хэшироваться с использованием bcrypt, при проверке пароля должна учитываться соль на основе логина. Сессия должна храниться в Redis и передаваться через cookie с флагами HttpOnly и SameSite Lax. Все операции с DM, серверами, топиками, сообщениями и звонками должны проверять права пользователя на серверной стороне; клиентские проверки не должны считаться достаточными.
- NF-7 **Поддерживаемость кода.** Серверная часть должна быть разделена на доменные модели, сервисы бизнес-логики, адаптеры к PostgreSQL, Redis и S3, HTTP handlers, WebSocket handlers и composition root. Клиентская часть должна быть разделена по пользовательским областям: auth, dm, server, chat и voice. Бизнес-логика не должна концентрироваться в cmd/main.go; этот файл должен отвечать за конфигурацию и сборку зависимостей.
- NF-8 **Наблюдаемость и диагностируемость.** Приложение должно логировать HTTP-запросы и ошибки серверной части. Ошибки WebSocket и WebRTC signaling должны возвращаться клиенту в структурированном виде через событие `error` или понятный ответ REST API. Сбор метрик времени ответа, числа активных WebSocket-соединений и активных звонков относится к дальнейшему развитию, но архитектура не должна препятствовать его добавлению.

Сформулированные требования устраняют недостаток отчёта по практике, где связь между анализом аналогов и проектированием была недостаточно явной. Таблица 3.1 показывает происхождение требований, а сами требования задают проверяемые пользовательские сценарии и технические ограничения. Далее эти требования используются как основание для описания архитектуры, модели данных, API, real-time механизма и проверки реализации.

4 Архитектура и проектирование

4.1 Общая архитектура

- описать Chatterly как клиент-серверное веб-приложение;
- указать основные внешние компоненты: браузер пользователя, Go backend, PostgreSQL, Redis, S3-совместимое хранилище, WebRTC/STUN-инфраструктура;
- объяснить поток обычного HTTP-запроса: frontend вызывает REST endpoint, API handler валидирует запрос, service выполняет бизнес-логику, adapter обращается к PostgreSQL, результат возвращается клиенту;
- объяснить поток real-time события: сообщение сохраняется в БД, backend публикует событие в Redis-канал пользователя, WebSocket session получает событие и отправляет его в браузер;
- указать, что `cmd/main.go` является composition root: создаёт конфигурацию, logger, подключения к Postgres/Redis/S3, transaction manager, adapters, stores, services, syncers, WebSocket manager и HTTP server.

4.2 Модель данных

- описать таблицу `users`: идентификатор, username, login, hashed password, avatar id, даты создания и обновления;
- описать таблицы личных сообщений: `dms`, `dm_participants`, `dm_messages`; объяснить, что `last_message_id` и `last_read_message_id` нужны для превью диалога и непочитанных сообщений;
- описать таблицы сообществ: `servers`, `server_participants`, `topics`, `topic_messages`;

- объяснить роли **owner** и **member**; владелец создаёт и изменяет структуру сервера, участник получает доступ к топикам;
- описать ограничения уникальности: уникальные логины и имена пользователей, уникальный участник в DM или server, уникальное имя топика одного типа внутри сервера;
- описать индексы сообщений DM по **dm_id**, **created_at** и **id**, так как история загружается постранично в обратном хронологическом порядке;
- в финальной версии добавить ER-диаграмму или схему таблиц, если её можно подготовить без перегрузки текста.

4.3 Backend-архитектура

- описать слой **internal/domain**: доменные структуры **User**, **DM**, **Server**, **Topic**, **TopicMessage**, **typed identifiers** и **value object Password**;
- описать слой **internal/api**: HTTP handlers для **user**, **image**, **dm**, **server**, **websocket** и **web**;
- описать слой **internal/service**: бизнес-логика пользователей, DM, серверов, текстовых топиков, изображений, **WebSocket sessions** и **voice topics**;
- описать слой **internal/adapter**: адаптеры **PostgreSQL**, **Redis** и **S3**, скрывающие детали внешних хранилищ от сервисов;
- описать **internal/store**: периодически синхронизируемые **in-memory** кэши пользователей, DM и серверов, используемые для быстрых **lookup** и поиска;
- описать **transaction manager**: операции создания DM, сообщений, серверов и топиков выполняются в транзакциях, чтобы связанные изменения сохранялись согласованно;
- объяснить, почему выбран **Go**: простая модель конкурентности, хорошая стандартная библиотека для HTTP, удобство построения сетевых сервисов;
- объяснить, почему выбран **chi**: лёгкий **router** и **middleware** для **request id**, **recovery**, **heartbeat** и **request logging**.

4.4 WebSocket и доставка сообщений

- описать endpoint **/ws/**, доступный только после аутентификации;

- описать жизненный цикл соединения: ассепт, создание **Connection**, регистрация в пользовательской session, запуск read pump и write pump, cleanup при закрытии;
- описать heartbeat: backend отправляет **ping**, frontend отвечает **pong**, отсутствие ответа приводит к закрытию соединения;
- описать подписку на каналы: клиент отправляет **join** с каналом, backend проверяет доступ к DM, text topic или voice topic;
- описать доставку сообщений: сервис DM или text topic публикует **message** в Redis-каналы всех участников, WebSocket manager доставляет событие активным соединениям пользователя;
- описать поведение frontend: singleton WebSocket store поддерживает переподключение, повторно отправляет **join** для активных каналов и буферизует события при временном отсутствии соединения.

4.5 WebRTC и голосовые топики

- описать назначение voice topic service: управление комнатами звонков внутри голосовых топики;
- описать выбор владельца комнаты через Redis key **voice:topic:<id>:owner**; если несколько backend-узлов обслуживают систему, события join/leave/signal перенаправляются владельцу комнаты;
- описать хранение активных участников через Redis sorted set **voice:topic:<id>:participant** и периодическое обновление TTL;
- описать signaling-события: offer, answer, ICE candidate и batch ICE candidates;
- описать использование библиотеки Pion WebRTC на backend: создание peer connection, регистрация кодеков, STUN server, настройка публичного IP и диапазона UDP-портов;
- описать forwarding медиаток: backend принимает remote track от участника, создаёт local track и добавляет его другим участникам комнаты;
- описать renegotiation при появлении или исчезновении треков, а также запрос ключевых кадров для восстановления видео;

- описать ограничения: без TURN server соединения могут быть нестабильны за строгими NAT; TURN следует вынести в перспективы развития.

4.6 Frontend-архитектура

- описать клиентскую часть как SolidJS-приложение на Vite;
- описать маршруты приложения: `/app/dm`, `/app/dm/search`, `/app/dm/:dmId`, `/app/server`, `/app/server/create`, `/app/server/manage`, `/app/server/:serverId/text/:topicId`, `/app/server/:serverId/edit`;
- описать разделение frontend по features: auth, dm, server, chat, voice;
- описать общий API-клиент: обработка JSON, сетевых ошибок, ошибок авторизации и ошибок backend;
- описать общий WebSocket store: единое соединение, подписчики событий, message handlers, error handlers, active channel map и pending events;
- описать chat component: загрузка первой страницы, догрузка истории при скролле вверх, дедупликация сообщений, автопрокрутка вниз, отправка сообщений;
- описать voice UI: local tile, remote tiles, статус звонка, меню микрофона/камеры/экрана, настройки media devices.

5 Реализация

5.1 Реализация backend

- описать реализацию конфигурации через переменные окружения: HTTP host/port, Postgres URL, Redis address, S3 endpoint, session parameters, chat message limit, image limits, voice settings;
- описать пользовательский сервис:
 - создание пользователя с bcrypt-хэшированием пароля;
 - вход по login/password;
 - создание сессии и запись session id в Redis;

- middleware, которая читает cookie, проверяет Redis и помещает user id в request context;
- обновление профиля с проверкой текущего пароля при изменении login или password;
- описать image service:
 - получение аватара по username;
 - генерация identicon, если пользователь не загрузил изображение;
 - проверка размера и разрешения загружаемого изображения;
 - конвертация изображения в JPEG и сохранение в S3-совместимое хранилище;
- описать DM service:
 - поиск пользователей без существующего DM;
 - создание DM с двумя участниками;
 - отправка сообщения, обновление последнего сообщения и отметка сообщения отправителя как прочитанного;
 - загрузка первой и следующих страниц истории;
 - проверка доступа участника перед чтением и подпиской;
- описать server service:
 - создание сервера и автоматическое добавление создателя как owner;
 - вступление и выход из сервера;
 - обновление и удаление сервера;
 - создание, обновление и удаление текстовых и голосовых топиков;
 - проверка роли owner при изменении структуры сервера;
- описать text topic service:
 - проверка, что отправитель состоит в сервере;
 - проверка, что топик имеет тип text;
 - сохранение сообщения в PostgreSQL;
 - публикация WebSocket-события всем участникам сервера;
- описать WebSocket manager:

- пользовательские sessions и несколько соединений одного пользователя;
- Redis subscription на канал пользователя;
- read pump для входящих событий;
- write pump для исходящих событий и heartbeat;
- cleanup с автоматическим выходом из voice topic;
- описать voice topic service:
 - join/leave пользователя в голосовой топик;
 - отправка `voice_state` новому участнику;
 - broadcast `voice_joined` и `voice_left`;
 - обработка offer, answer и ICE candidates;
 - forwarding RTP-треков между участниками;
 - закрытие peer connection и удаление треков при выходе участника.

5.2 Реализация frontend

- описать страницы входа и регистрации: отдельные endpoints для `login.html` и `signup.html`, вызовы API создания пользователя и входа;
- описать главный app shell: router с base path `/app`, провайдеры, глобальные DM-уведомления и toast storage;
- описать DM UI:
 - список личных диалогов в sidebar;
 - поиск пользователей;
 - создание DM из результата поиска;
 - обновление preview и unread состояния при входящем WebSocket-сообщении;
- описать server UI:
 - список серверов пользователя;
 - создание сервера;
 - поиск и присоединение к серверам;

- управление сервером и топиками;
- переходы в text topic и voice topic;
- описать chat UI:
 - единый компонент для DM и text topic;
 - REST-загрузка истории;
 - подписка на WebSocket-канал при открытии чата;
 - отправка сообщений через REST API;
 - обработка live-сообщений и догрузка старых сообщений;
- описать voice UI:
 - автоматический join в voice topic при открытии страницы;
 - создание `RTCPeerConnection`;
 - синхронизация local tracks при включении микрофона, камеры или экрана;
 - обработка server offer/answer/ICE;
 - отображение удалённых участников и локального preview;
 - выбор камеры, микрофона и speaker device, если браузер поддерживает `setSinkId`.
- отдельно описать текущие ограничения frontend: profile settings modal не должен быть заявлен как полностью завершённый, если к финальной версии не подключены реальные endpoint-ы обновления профиля и аватара.

5.3 Локальная инфраструктура

- описать `docker-compose.local.yaml`: PostgreSQL, Redis, MinIO и вспомогательный контейнер создания bucket;
- описать Makefile-команды: `make up`, `make run`, `make run-web`, `make build`, `make build-web`, `make test`;
- описать миграции Goose и генерацию SQLC-кода из `queries/*.sql` и `migrations/*.sql`;
- указать, что production deploy находится вне текущего реализованного MVP, если не будет завершён до сдачи.

6 Тестирование и оценка результата

6.1 Подход к тестированию

- написать, что для программного проекта требуется не только описать реализацию, но и показать корректность ключевых пользовательских сценариев;
- разделить проверки на несколько уровней: сборка, backend API, real-time события, frontend сценарии, звонки, безопасность доступа, производительность;
- честно указать состояние автотестов: в репозитории есть команда `go test ./...`, но unit/e2e тесты для сервисов и API handlers отмечены как незавершённые; если к финалу они будут добавлены, заменить этот пункт на описание конкретных тестов;
- привести таблицу сценариев тестирования, где для каждого сценария указать входные данные, ожидаемый результат и фактический статус.

6.2 Сценарии функционального тестирования

- **Регистрация и вход.** Создать пользователя, войти с правильным паролем, убедиться, что `/v1/user/me` возвращает текущий профиль; проверить ошибку при неверном пароле.
- **Сессии.** Проверить, что защищённые endpoints возвращают ошибку без cookie и работают после входа; проверить logout и невозможность повторного доступа по очищенной сессии.
- **Профиль и аватар.** Проверить получение identicon для пользователя без аватара, загрузку изображения допустимого размера, отказ при слишком большом файле или некорректном формате.
- **Личные сообщения.** Создать двух пользователей, найти второго пользователя, создать DM, отправить сообщение, проверить появление сообщения у обоих пользователей и обновление preview.
- **История DM.** Создать больше сообщений, чем `MESSAGES_LIMIT`, загрузить первую страницу и следующую страницу по cursor, проверить порядок и отсутствие дублей.
- **Серверы и топики.** Создать сервер, убедиться, что создатель получил роль owner, создать text topic и voice topic, изменить и удалить топик.

- **Права доступа.** Проверить, что пользователь, не состоящий в сервере, не может читать сообщения топика и подписываться на WebSocket-канал этого топика; проверить, что member не может изменять структуру сервера, если такое ограничение реализовано.
- **Текстовый топик.** Отправить сообщение в text topic и проверить его доставку всем участникам сервера через WebSocket.
- **WebSocket reconnect.** Разорвать соединение, дождаться переподключения frontend, проверить повторный join активного канала и получение новых сообщений.
- **Голосовой топик.** Открыть voice topic двумя пользователями, проверить voice_state, отображение участников, передачу микрофона, камеры и отключение участника при закрытии вкладки.
- **Демонстрация экрана.** Включить screen share, проверить появление видеопотока у другого участника и корректное прекращение трансляции.

6.3 Оценка производительности

- описать целевую метрику для чата: задержка от отправки сообщения одним пользователем до отображения у других участников;
- описать целевую метрику для WebSocket: число активных соединений, стабильность heartbeat, отсутствие массовых reconnect при десяти пользователях;
- описать целевую метрику для звонка: успешность подключения, время до появления удалённого участника, устойчивость аудио/видео при двух и более участниках;
- зафиксировать тестовую конфигурацию: машина, браузер, число пользователей, способ эмуляции пользователей, параметры PostgreSQL/Redis/backend;
- если нагрузочное тестирование не будет проведено, написать это как ограничение и оставить только ручную проверку сценариев MVP.

6.4 Оценка соответствия требованиям

- составить таблицу соответствия: требование, реализованный компонент, способ проверки, статус;

- для реализованных требований ссылаться на разделы реализации: user service, DM service, server service, text topic service, WebSocket manager, voice topic service, frontend routes;
- для частично реализованных требований указать причину и план завершения, например profile settings modal, автотесты, метрики, production deploy, TURN server;
- в выводе раздела написать, какие сценарии MVP подтверждены, а какие остаются перспективами развития.

7 Заключение

План итогового текста заключения:

- вернуться к цели из введения и написать, что в работе был спроектирован и реализован MVP веб-приложения Chatterly для группового общения;
- перечислить результаты:
 - проведён анализ Discord, Zoom, Jitsi Meet, Matrix / Element и TeamSpeak;
 - сформированы функциональные и нефункциональные требования;
 - спроектирована архитектура backend, frontend, хранилищ, WebSocket и WebRTC;
 - реализованы пользователи, сессии, профили, аватары, серверы, топики, личные сообщения, история сообщений и доставка событий в реальном времени;
 - реализованы голосовые топики с WebRTC signaling, локальными media controls и отображением участников;
 - подготовлена локальная инфраструктура разработки на Docker Compose;
- описать практическую полезность: приложение демонстрирует независимую браузерную платформу для сообществ, объединяющую текстовое общение и звонки;
- указать ограничения честно и конкретно: отсутствие production deploy, неполное автотестирование, необходимость TURN server для сложных сетей, необходимость расширить роли и модерацию, необходимость подключить или доработать все сценарии профиля, если они не будут завершены;

- описать направления дальнейшего развития: e2e и unit tests, мониторинг и метрики, нагрузочное тестирование, мобильная адаптация, расширенные права доступа, push-уведомления, файлы в чатах, TURN-инфраструктура.

Список литературы

- [1] *About Element*. Element. URL: <https://element.io/about> (дата обр. 21.04.2026).
- [2] *About Jitsi Meet*. Jitsi. URL: <https://jitsi.org/jitsi-meet/> (дата обр. 19.04.2026).
- [3] *Creating and using Zoom Chat channels*. Zoom. URL: https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0063548 (дата обр. 07.05.2026).
- [4] *Discord Server Setup Guide*. Discord. 2 июля 2025. URL: <https://support.discord.com/hc/en-us/articles/33023827550359-Discord-Server-Setup-Guide> (дата обр. 20.04.2026).
- [5] *How to voice chat*. TeamSpeak. 14 апр. 2026. URL: <https://support.teamspeak.com/hc/en-us/articles/35367762165405-How-to-voice-chat> (дата обр. 21.04.2026).
- [6] *Matrix Specification*. Matrix.org. URL: <https://spec.matrix.org/> (дата обр. 21.04.2026).
- [7] *Screen sharing software*. Zoom. URL: <https://www.zoom.com/en/products/virtual-meetings/features/screen-sharing/> (дата обр. 19.04.2026).
- [8] *Secure collaboration platform for enterprises*. Element. URL: <https://element.io/solutions/secure-collaboration> (дата обр. 21.04.2026).
- [9] Евгений Одинцов. *В России предрекли скорую блокировку Zoom*. Газета.Ру. 10 сент. 2025. URL: <https://www.gazeta.ru/social/news/2025/09/10/26692490.shtml> (дата обр. 21.04.2026).
- [10] *Роскомнадзор заблокировал Discord*. РБК. 8 окт. 2024. URL: https://www.rbc.ru/technology_and_media/08/10/2024/67054cbf9a79474670135b84 (дата обр. 22.04.2026).